

Query Processing

Merge Join (Sort-Merge Join)

1. Introduction

The **Merge Join** algorithm (also called **Sort-Merge Join**) computes natural joins and equi-joins efficiently when both input relations are **sorted on the join attributes**.

Key Idea: Similar to the merge phase in merge-sort — scan both sorted relations simultaneously, matching tuples with equal join attribute values.

Key Terminology

- **JoinAttrs:** The common attributes $R \cap S$ used for joining
- S_s : Set of tuples from s with the same join attribute value (must fit in memory)
- b_r, b_s : Number of blocks in relations r and s
- b_b : Number of buffer blocks allocated to each relation
- p_r, p_s : Pointers to current tuples in r and s

1.1 Running Example

Relation	Tuples	Blocks
student	5,000	$b_{student} = 100$
takes	10,000	$b_{takes} = 400$

Join attribute: ID

2. Merge-Join Algorithm

Precondition: Both relations must be sorted on the join attributes.

Algorithm: Merge Join

```
p_r = first tuple of r
p_s = first tuple of s

while (p_r != null AND p_s != null):
    t_s = tuple at p_s
    S_s = {t_s} // Collect all tuples with same join value

    // Gather all s-tuples with same join attribute value
    advance p_s
    while (p_s != null AND p_s[JoinAttrs] == t_s[JoinAttrs]):
        add tuple at p_s to S_s
        advance p_s

    // Skip r-tuples with smaller join values
    while (p_r != null AND p_r[JoinAttrs] < t_s[JoinAttrs]):
        advance p_r

    // Match all r-tuples with same join value against S_s
    while (p_r != null AND p_r[JoinAttrs] == t_s[JoinAttrs]):
        for each tuple t_s in S_s:
            output join(p_r, t_s)
        advance p_r
```

2.1 Visual Example

Consider two sorted relations on join attribute **a1**:

Relation r (sorted)	Relation s (sorted)
a1=1, a2=x	a1=1, b1=p
a1=2, a2=y	a1=1, b1=q
a1=3, a2=z	a1=3, b1=r
a1=4, a2=w	a1=5, b1=s

Merge Process:

1. p_r at a1=1, p_s at a1=1: Collect $S_s = \{(1, p), (1, q)\}$
2. Match r 's (1,x) with both $\rightarrow$ Output: (1,x,p), (1,x,q)
3. Advance p_r to a1=2; p_s now at a1=3
4. a1=2 ; a1=3: Skip r 's (2,y) — no match
5. p_r at a1=3, p_s at a1=3: Match $\rightarrow$ Output: (3,z,r)
6. Continue until both exhausted

3. Cost Analysis

3.1 When Relations Are Already Sorted

Cost Formula: Merge Join (Pre-Sorted)

$$\text{Block Transfers} = b_r + b_s$$

$$\text{Seeks} = \left\lceil \frac{b_r}{b_b} \right\rceil + \left\lceil \frac{b_s}{b_b} \right\rceil$$

Where b_b = buffer blocks allocated per relation.

Why so efficient?

- Each tuple is read **exactly once**
- Each block is read **exactly once**
- Single pass through both files

Example: Pre-Sorted Relations

student ⋈ **takes** (both sorted on ID):

$$\text{Block Transfers} = 100 + 400 = \mathbf{500}$$

With $b_b = 1$ (**worst case**):

$$\text{Seeks} = 100 + 400 = \mathbf{500}$$

With larger buffers: Seeks decrease significantly!

3.2 Optimal Buffer Allocation

Rule of Thumb: Since seeks are expensive ($t_S = 4$ ms vs $t_T = 0.1$ ms), allocate multiple buffer blocks to each relation.

Example: With $b_b = 400$ blocks (1.6 MB per relation):

- Transfer time for 400 blocks: $400 \times 0.1 = 40$ ms
- Seek time: 4 ms
- Seek overhead: only 10% of total I/O time

4. When Relations Are NOT Sorted

If relations are unsorted, we must **sort them first**, then merge join.

Total Cost = Cost to Sort r + Cost to Sort s + Cost of Merge Join

4.1 Case 1: Small Memory ($M = 3$ blocks)

Detailed Calculation: $M = 3$ blocks

Step 1: Sort takes ($b_{takes} = 400$)

Number of merge passes:

$$P = \left\lceil \log_{M-1} \left(\frac{b_{takes}}{M} \right) \right\rceil = \left\lceil \log_2 \left(\frac{400}{3} \right) \right\rceil = \lceil 7.06 \rceil = 8 \text{ passes}$$

Block transfers for sorting:

$$400 \times (2 \times 8 + 1) = 400 \times 17 = 6800 \text{ transfers}$$

Plus 400 transfers to write sorted output.

Seeks for sorting:

$$2 \times \left\lceil \frac{400}{3} \right\rceil + 400 \times (2 \times 8 - 1) = 268 + 6000 = 6268 \text{ seeks}$$

Plus 400 seeks to write output = **6668 seeks total**.

Step 2: Sort student ($b_{student} = 100$)

Merge passes: $\lceil \log_2(100/3) \rceil = 6$

Block transfers: $100 \times (2 \times 6 + 1) = 1300 + 100 \text{ write} = \mathbf{1400}$

Seeks: $2 \times \lceil 100/3 \rceil + 100 \times (2 \times 6 - 1) = 68 + 1100 = 1168 + 100 = \mathbf{1268}$

Step 3: Merge Join

$$\text{Transfers} = 400 + 100 = 500 \quad \text{Seeks} = 500$$

Total Cost ($M = 3$):

Phase	Transfers	Seeks
Sort takes	7200	6668
Sort student	1400	1268
Merge Join	500	500
Total	9100	8436

4.2 Case 2: Larger Memory ($M = 25$ blocks)

Detailed Calculation: $M = 25$ blocks

Step 1: Sort takes

Merge passes: $\lceil \log_{24}(400/25) \rceil = \lceil \log_{24}(16) \rceil = 1$

Block transfers: $400 \times (2 \times 1 + 1) = 1200 + 400 \text{ write} = \mathbf{1600}$

Step 2: Sort student

Block transfers: $100 \times 3 = 300 + 100 \text{ write} = \mathbf{400}$

Step 3: Merge Join

Transfers: 500

Total Cost ($M = 25$):

Phase	Transfers	Seeks
Sort takes	1600	832
Sort student	400	208
Merge Join	500	500
Total	2500	1540

With optimized buffer allocation ($b_b = 12$ for merge):

$$\text{Merge Seeks} = \left\lceil \frac{400}{12} \right\rceil + \left\lceil \frac{100}{12} \right\rceil = 34 + 9 = 43$$

Total Seeks reduced to ≈ 251 !

4.3 Memory Impact Summary

Memory	Transfers	Seeks	Time (est.)
$M = 3$	9,100	8,436	≈ 34.7 sec
$M = 25$	2,500	1,540	≈ 6.4 sec
$M = 25$ (optimized)	2,500	251	≈ 1.25 sec
Pre-sorted	500	2	≈ 0.06 sec

5. Hybrid Merge Join

Used when one relation is **sorted** and the other has a **secondary B⁺-tree index** on the join attribute.

Algorithm: Hybrid Merge Join

1. Merge the sorted relation with the LEAF ENTRIES of the secondary index (not the actual tuples)
2. Result: tuples from sorted relation + ADDRESSES of matching unsorted tuples
3. Sort the result file by the tuple addresses
4. Fetch unsorted tuples in PHYSICAL STORAGE ORDER (sequential I/O instead of random)

Why is this efficient?

- Avoids random I/O for each tuple lookup
- Fetches unsorted tuples in physical order (sequential I/O)
- Trades one sort operation for many random seeks

Hybrid Merge Join Example

Scenario:

- **student** is sorted on ID
- **takes** is unsorted but has a secondary B⁺-tree on ID

Process:

1. Scan **student** and B⁺-tree leaves of **takes** together
2. For each match: output (student tuple, takes RID)
3. Sort result by takes RID (physical address)
4. Fetch takes tuples in sorted address order

Benefit: Instead of 10,000 random I/Os for takes, we do one sequential pass after sorting addresses.

When to use Hybrid Merge Join:

- One relation is already sorted (or cheap to sort)
- Other relation has a secondary index but is large/unsorted
- Random I/O cost would be prohibitive for indexed nested-loop

6. Summary

Scenario	Cost	Notes
Both sorted	$b_r + b_s$ transfers	Optimal case
Need to sort	$+O(b \log b)$ per relation	Use external merge sort
Hybrid (1 sorted, 1 indexed)	Merge + sort addresses	Avoids random I/O

Key Takeaways

- **Merge Join** is highly efficient for sorted relations: $O(b_r + b_s)$
- **Single pass** through both relations — each block read once
- If unsorted: **sort first**, then merge (sort cost dominates)
- **More memory** = fewer merge passes = dramatically lower cost
- **Buffer allocation** for seeks is critical ($b_b > 1$ recommended)
- **Hybrid Merge Join** trades extra sorting for sequential I/O
- Requirement: Each set S_s (tuples with same join value) must fit in memory